Database Users, Roles, Privileges, and Access Control

Group 3 (On-the-Job Training (OJT) Record Management Application)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** On-the-Job Training (OJT) Record Management Application **Repository Link: https://github.com/swvenz/Project-OJT-RMA-System.git** **Our Group Repository:**
Projects set up	**The OJT RMA System repository was used as the basis for preparing the database access-control plan. The project files were reviewed to understand the application's database requirements and identify the database objects that need appropriate access restrictions.** **The first step was to examine the existing project files, including the database configuration, SQL scripts, and other files related to database connectivity. This process helps identify the database structure and determine which users or application components may need access to the database.** **The database used for this activity was named [ENTER ACTUAL DATABASE NAME]. The existing database structure was retained, and the access-control implementation was prepared separately to avoid changing the original system functionality.** **The following database objects were identified during the database assessment:** • **[Table name 1] – [Describe the purpose of the table.]** • **[Table name 2] – [Describe the purpose of the table.]** • **[Table name 3] – [Describe the purpose of the table.]** • **[Table name 4] – [Describe the purpose of the table.]** • **[Additional tables] – [Describe their purposes.]** **After reviewing the database structure, a separate folder named `access_control` was prepared to organize the SQL scripts and access-control documentation.** **The following files were prepared:** • `01_create_roles.sql`

	• `02_grant_privileges.sql` • `03_verify_permissions.sql` • `access_control_plan.md` **These files were organized to make the implementation easier to understand, maintain, and verify.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each role. The implementation follows the principle of least privilege, meaning that each role should only receive the permissions needed for its responsibilities.** **Three PostgreSQL group roles were proposed for the OJT RMA System.** **Administrator Role (`rma_admin_grp`)** **The administrator role is responsible for managing and maintaining the database. It is intended for authorized personnel who need to manage database permissions and perform administrative activities.** **Its responsibilities include maintaining database records, managing database access, and performing authorized maintenance operations.** **Runtime Role (`rma_runtime_grp`)** **The runtime role is intended for normal application operations. It allows the application to retrieve, add, update, and delete database records when performing its required functions.** **Its responsibilities include accessing the application's records, processing the operations supported by the system, and maintaining information required by the application.** **Read-only Role (`rma_readonly_grp`)** **The read-only role is intended for users who only need to view database information. It receives SELECT privileges but cannot insert, update, or delete records.** **Its purpose is to allow authorized users to view information without making unauthorized changes to the database.**

<table>
<tr>
<td></td>
<td>
Access-Control Matrix
<table>
<tr><th>Database Operations</th><th>Admin</th><th>Runtime</th><th>Read-only</th></tr>
<tr><td>SELECT</td><td>Allowed</td><td>Allowed</td><td>Allowed</td></tr>
<tr><td>INSERT</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>UPDATE</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>DELETE</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>Sequence Access</td><td>Full required access</td><td>Required usage access</td><td>Not granted</td></tr>
</table>
The three group roles were designed with the <code>NOLOGIN</code> attribute. This means they function as permission groups and must be assigned to actual login accounts before individual users or application accounts can use their permissions.

The actual privileges should be adjusted according to the database objects and operations required by the OJT RMA System.
</td>
</tr>
<tr>
<td>SQL scripts summary</td>
<td>
Three SQL scripts were prepared to implement and verify the access-control configuration.

File 1: <code>01_create_roles.sql</code>

The first SQL script is used to create the three PostgreSQL group roles.

The script checks whether each role already exists before creating it. This helps prevent errors when the script is executed more than once.
</td>
</tr>
</table>

	The following roles are created: • **`rma_admin_grp`** • **`rma_runtime_grp`** • **`rma_readonly_grp`** **After executing the script through pgAdmin 4, a verification query is used to display the created roles from the PostgreSQL system catalog.** **File 2: `02_grant_privileges.sql`** **The second SQL script is used to assign the required database permissions to each role.** **First, the roles are given permission to connect to the OJT RMA System database and access the required schema.** **The administrator role receives the necessary administrative privileges on the database objects. The runtime role receives SELECT, INSERT, UPDATE, and DELETE permissions on the tables required by the application, together with the necessary sequence permissions.** **The read-only role receives SELECT privileges only.** **The script may also include default privileges for future tables and sequences. This helps maintain consistent permissions when new database objects are created by the specified owner.** **File 3: `03_verify_permissions.sql`** **The third SQL script is used to check whether the assigned permissions are correct.** **The script contains queries that display the created roles, list their table privileges, and test specific permissions using PostgreSQL's `has_table_privilege()` function.** **The verification focuses on checking whether:** • **The administrator role can delete records.** • **The runtime role can insert records.** • **The runtime role can update records.** • **The read-only role can select records.** • **The read-only role cannot insert records.** • **The read-only role cannot update or delete records.**

	These checks help confirm that each role follows the intended access-control plan.
Verification results	**After preparing the three SQL scripts, they should be executed in the PostgreSQL Query Tool through pgAdmin 4.** **The scripts should be executed in the correct order to avoid permission-related errors.** **First Verification: Role Creation** **The first script is executed to create the three group roles. The PostgreSQL system catalog is then queried to confirm that the roles exist.** **Expected result: All three roles should appear in the query results.** **Second Verification: Privilege Assignment** **The second script is executed to assign the required database, schema, table, and sequence privileges. The execution messages are checked to determine whether the commands completed successfully.** **Expected result: Each role should receive the permissions defined in the access-control plan.** **Third Verification: Permission Checking** **The third script is executed to verify the permissions using PostgreSQL queries.** **The expected results are:** **Verification** — **Expected Result** **Admin can delete records** — **TRUE**

Runtime can insert records	**TRUE**
Runtime can update records	**TRUE**
Read-only can select records	**TRUE**
Read-only can insert records	**FALSE**
Read-only can update records	**FALSE**
Read-only can delete records	**FALSE**

The expected results demonstrate that the administrator and runtime roles have their intended database permissions, while the read-only role is restricted to viewing records.

Screenshots:

- **Screenshot 1: `01_create_roles.png`**
- **Screenshot 2: `02_grant_privileges.png`**
- **Screenshot 3: `03_verify_permissions.png`**

The screenshots should serve as supporting evidence of the SQL execution and verification process. Actual test results must be recorded after executing the scripts.

Issues and fixes	**• During the preparation and implementation of the access-control configuration, the following possible issues were considered.** **• Database Name Mismatch: The SQL scripts must be executed inside the correct database. The actual OJT RMA System database name should be used consistently to avoid applying the configuration to the wrong database.** **• Existing Database Roles: Running role creation commands repeatedly may cause errors if the roles already exist. The first SQL script uses an existence check before creating each role.** **• Incorrect Execution Order: Granting permissions before creating the roles can result in errors. To prevent this, the scripts are executed in order, starting with role creation, followed by privilege assignment, and then verification.** **• Insufficient Database Permissions: Some PostgreSQL commands require appropriate administrative privileges. The scripts should be executed using an authorized database administrator account to ensure that the necessary permissions are available.** **• Future Database Objects: Newly created tables and sequences may not automatically receive the same permissions as existing objects. Default privileges should be reviewed and configured for the appropriate object owner.** **• Read-only Permission Restrictions: The read-only role should not be able to modify database records. The verification script checks that SELECT is allowed while INSERT, UPDATE, and DELETE are denied.**
Feedback to owners	**The OJT RMA System repository was reviewed as the basis for the database access-control activity. Implementing role-based access control can help improve the security and organization of database operations.** **The following recommendations are provided to the project owners:** **• Maintain separate database roles for administrators and normal application operations.** **• Avoid using administrator-level database credentials for regular application activities.** **• Assign actual login accounts to the appropriate group roles.** **• Use strong passwords and protect database credentials.** **• Review database permissions whenever new tables or features are added.** **• Restrict access to sensitive application information.**

	• **Regularly check database roles and privileges to prevent unnecessary access.** **These recommendations can help improve the security and maintainability of the OJT RMA System.**
AI usage declaration	**Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation.** **AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, prepare the access-control plan, and identify possible implementation issues and their corresponding solutions.** **The generated SQL scripts and proposed permissions must be reviewed against the actual OJT RMA System database structure. The scripts should be executed and checked using PostgreSQL through pgAdmin 4. The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.** # Conclusion **The Activity 4 implementation prepared a role-based access-control structure for the OJT RMA System using PostgreSQL. Three group roles were proposed to separate administrative operations, normal application activities, and read-only access.** **Through the creation of roles, assignment of privileges, and verification of permissions, the activity demonstrates how database access can be managed according to user responsibilities.** **Following the principle of least privilege helps protect important application records and provides a more organized approach to database security. The SQL scripts and verification evidence should be maintained in the group's own repository as part of the Activity 4 submission.**
